

Tools for software development

Test management



Testing with experience

- black box – design from knowledge about the system
- white box – design from documentation and source code
- The tester/analyst has sufficient experience with:
 - by testing
 - software deployment area
 - understands the background of the problem being solved (design, architecture...)
 - can understand the requirements and their meaning just like the customer/user
- **Experience-based testing**
 - problematic for complex systems
 - necessary knowledge of the issue (banking, telecommunications, medicine...)

Experience-based tests

- can be implemented in the case of:
 - unless documentation is available (*or documentation is poor*)
 - we have an experienced tester
- it is always necessary to carry out another type of testing
- When is it functional:
 - legitimate reasons – incorrect estimates, limited budget resources
 - significant reduction in product testing time
 - only for experienced testers and analysts
- Approaches:
 - Ad hoc testing
 - Exploratory *testing*

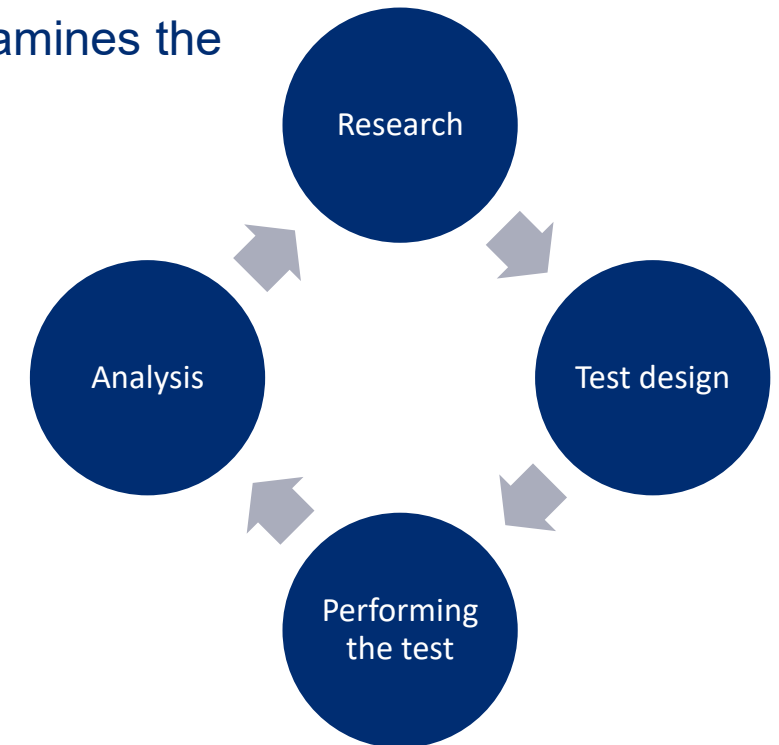
Ad hoc testing

- unsystematic, unplanned and randomly performed activity, the purpose of which is to discover as many defects as possible
- usually no documentation is done
- the application's test coverage is low
- we refer to as ineffective testing without a strategy
- but if chosen as a supplementary test format it can be effective



Exploratory testing

- often considered the same as ad hoc
- we can talk about the approach to testing
- systematic and thorough execution of ad hoc testing, where the tester examines the product and, based on this, designs and executes test cases
- investigation:
 - mapping of functions, data, identification of potential problem areas
 - The tester learns to use the product and on this basis is able to design the right strategy for evaluating its behavior – creating test cases
- Unlike specification-based tests, this is an interactive process
 - the tester proposes test cases based on the functionality being investigated
 - can lead to rapid detection of problem areas
- result is, in addition to a list of defects, a system review and a proposal for its testing



Exploratory testing

Example:

- The tested software allows you to perform two different functionalities 1 and 2
- both functionalities have very similar GUIs
- tests according to specification work without problems
- In exploratory testing, the tester notices the similarity of these two GUIs
- there is a suspicion that they are operating on the same instance
- by verifying the suspicion - testing - the use of one instance is confirmed
- Further testing reveals that when functionality 2 was interrupted and functionality 1 was switched to, some data in the instance was not deleted and caused functionality 1 to fail.



Exploratory testing

Recommendations:

- Before testing, it is necessary to decide, based on a risk analysis, which areas we will focus on during testing - there is never enough time for a complete retest
- Sometimes it is useful to focus on one characteristic of a system – testing functionality, reliability, security...
- It is inappropriate to start with complex functionality unless we have enough information about the system at the beginning, and we will acquire this information through gradual research.
- with sufficient knowledge of the system, starting with complex functionality can be beneficial, because we will find significant problems here, more detailed testing is unnecessary
- if we get stuck in an area, it is necessary to jump to the next area - often the next area is the solution to the previous problem
- let's not forget the necessary notes

Exploratory testing

Measurement and control:

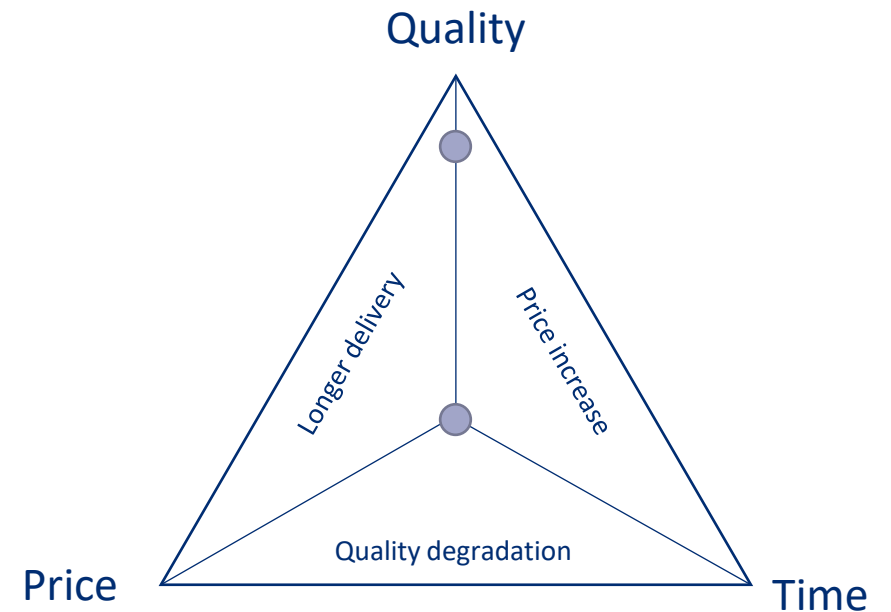
- freedom and absence of documentation make it difficult to evaluate the tester
- the possibility lies in applying *session-based test management*
- *session* – a period of time dedicated only to testing, which ends with a report
- Report format:
 - goal and area of testing
 - date, time and tester identification
 - task distribution – percentage expression of the ratio of time between testing, research, error investigation and report creation
 - data – inputs, outputs
 - notes – what we tested, how and what I plan to do next
 - problems – are there any questions or ambiguities?
 - detected defects

Test management

- Testing structure and processes are usually defined in the form of documentation and distributed within the company to people who come into contact with testing.
- is defined in particular:
 - Testing principles
 - philosophy, especially goals and overall approach to testing in the company
 - Testing strategy
 - requirements and methods for conducting testing
 - Master test plan
 - focuses specifically on the project and describes in which situation which strategy will be applied
 - Plan for each level of testing
 - for larger projects, describes plans for each level with details for that level (unit tests, integration tests)
- The scope, structure, content and form change according to the needs of the environment.
 - briefer notes
 - necessary information

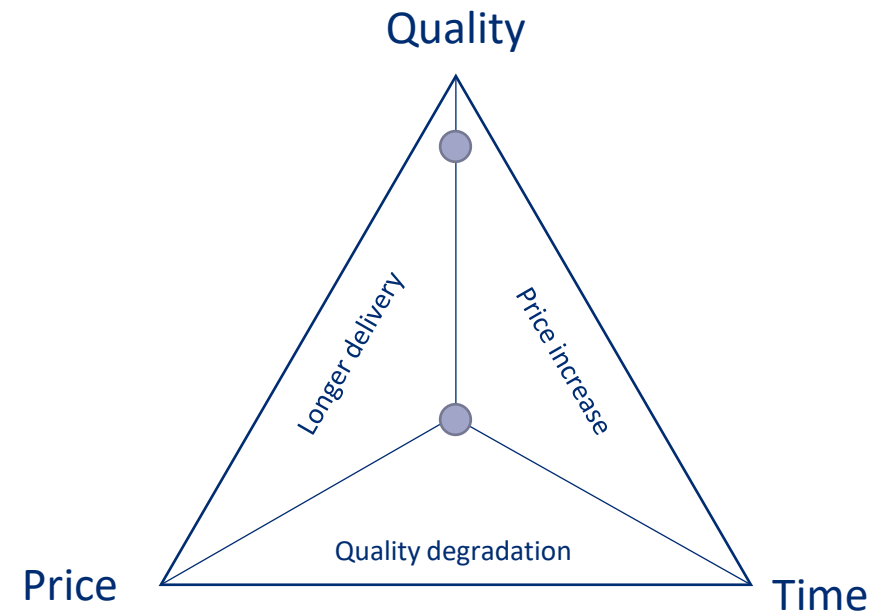
Testing planning

- we are based on the requirements and needs of the " *stakeholder* "
- basic attributes:
 - delivery time
 - price
 - quality of the resulting product
- the ideal is unattainable - finding a compromise
- quality triangle
 - defines maintaining a balance between attributes
 - if we change the values of one, it will affect the other two
 - e.g. cost reduction must be reflected in reduced quality and/or delivery time



Testing planning

- we are based on the requirements and needs of the " *stakeholder* "
- basic attributes:
 - delivery time - immediately
 - price - free
 - quality of the resulting product - high quality
- the ideal is unattainable - finding a compromise
- quality triangle
 - defines maintaining a balance between attributes
 - if we change the values of one, it will affect the other two
 - e.g. cost reduction must be reflected in reduced quality and/or delivery time



Test plan

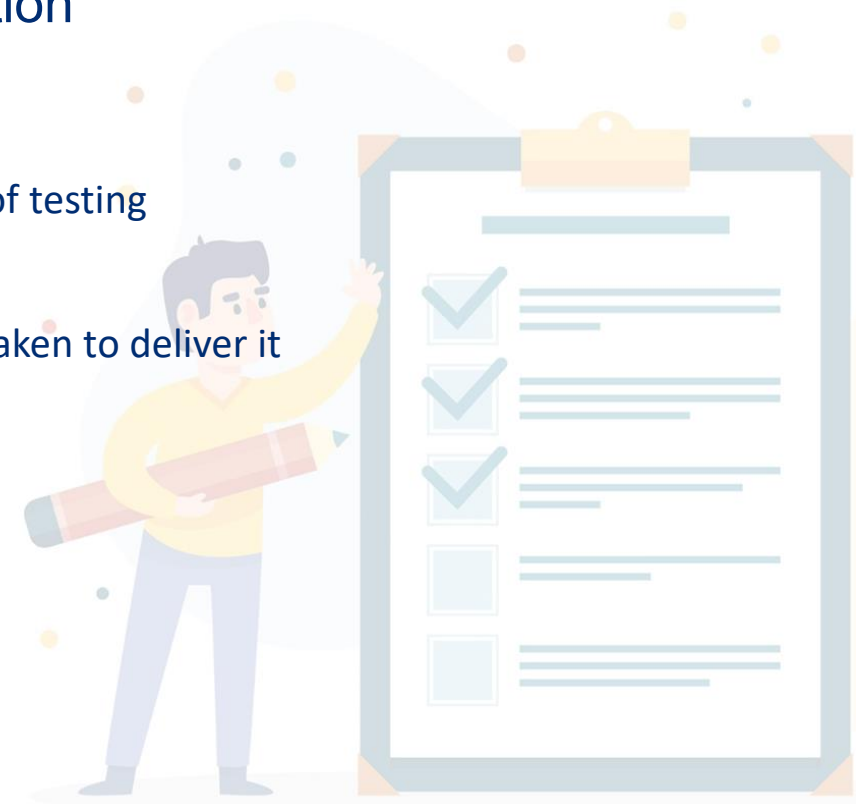
The purpose of the document is to define:

- **Testing goal** – intention and goal (*proving functionality, meeting requirements, debugging defects*)
- **Testing framework** – a clear definition of what is being tested
- **Testing method and strategy** – how will testing be carried out (*methods, tools, approach and technique*)
- **Risks of the testing system** – changes in the system, poor specification, API
- **Resources needed** – hardware, people, environmental requirements
- **Test artifacts** – test cases, test data, scripts, recorded defects, test reports
- **Planned activities** – testing schedule, tester tasks
- **Testing schedule** – from analysis, test case design to evaluation of test results
- **Metrics**
- **Identification of exit criteria** – definition of conditions under which their fulfillment is a prerequisite for the completion of a certain test activity or plan
- **Identifying criteria for pausing and resuming testing** – when to pause testing (many defects discovered) or resume testing.
- **Risks threatening testing** – budget, resources, changes in the timetable

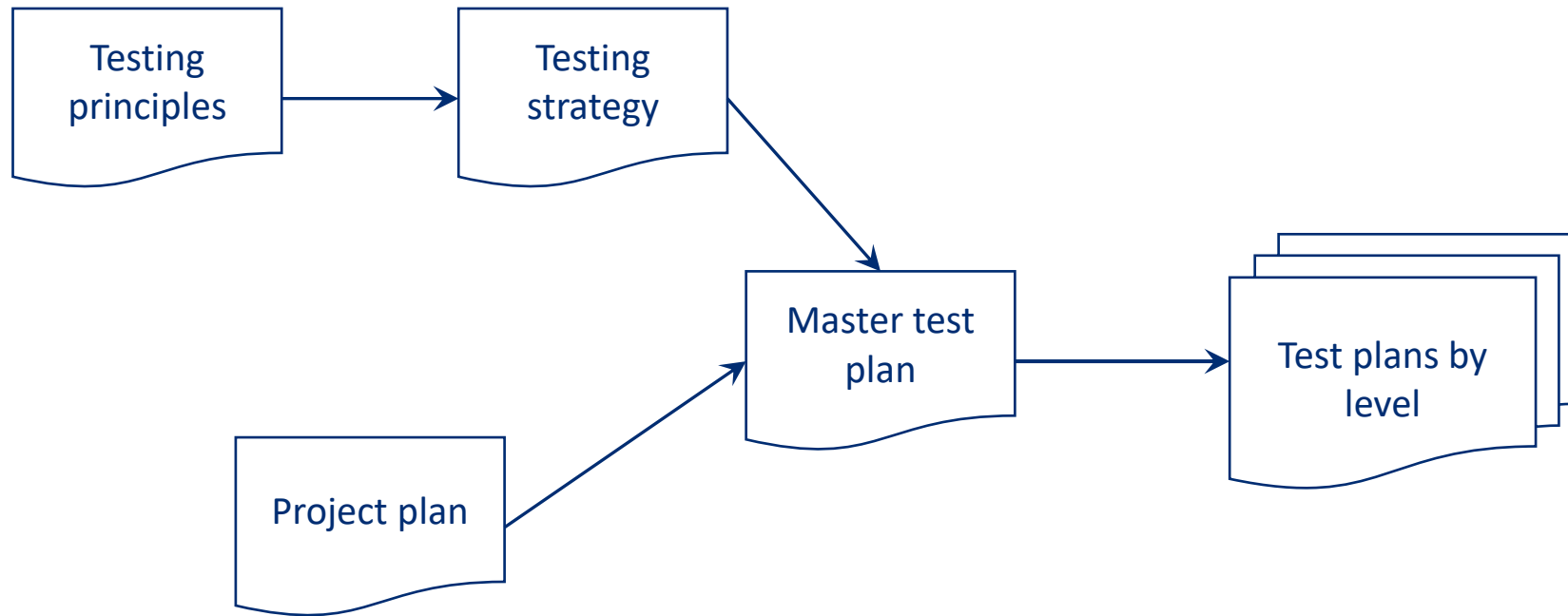


Creating a test plan

- in good time before
- we can base our work on the Standards for Software Testing Documentation: **IEEE 829:2008**
- when creating, we usually don't have all the necessary information
- as we prepare for each level
 - the possibility of including in the specification stage a plan of acceptance tests, which are the output criteria and are also the final conditions for the completion of testing
- in case of missing, incomplete information
 - noting in the plan when this information will be delivered and what steps will be taken to deliver it
 - delivery date + name of responsible person
- the completed plan must be approved by the stakeholder
- additional changes during the software development cycle

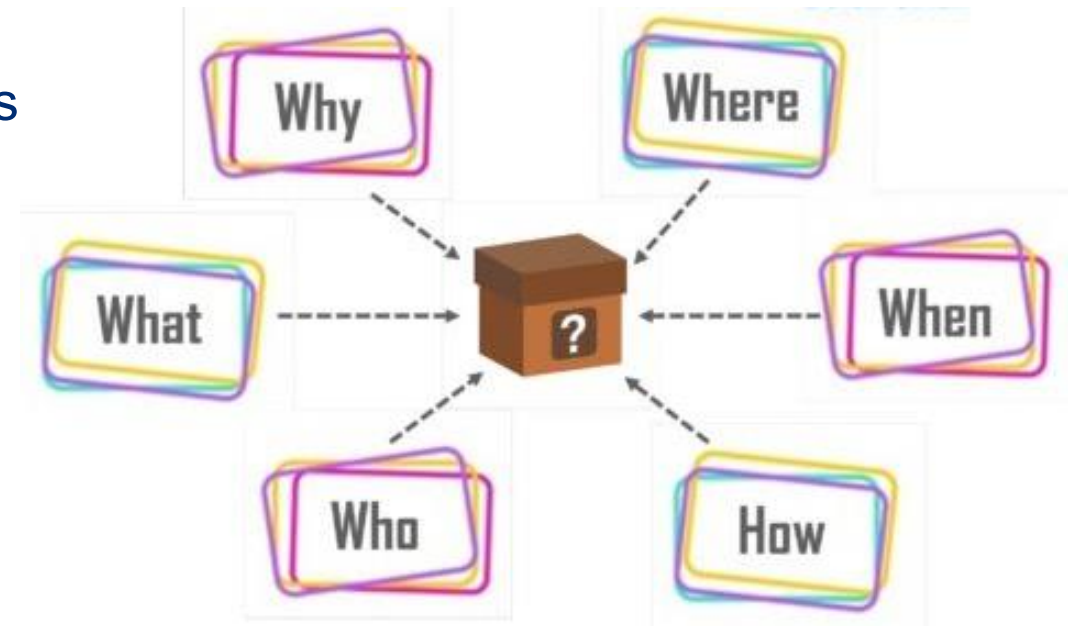


Plan inputs and outputs

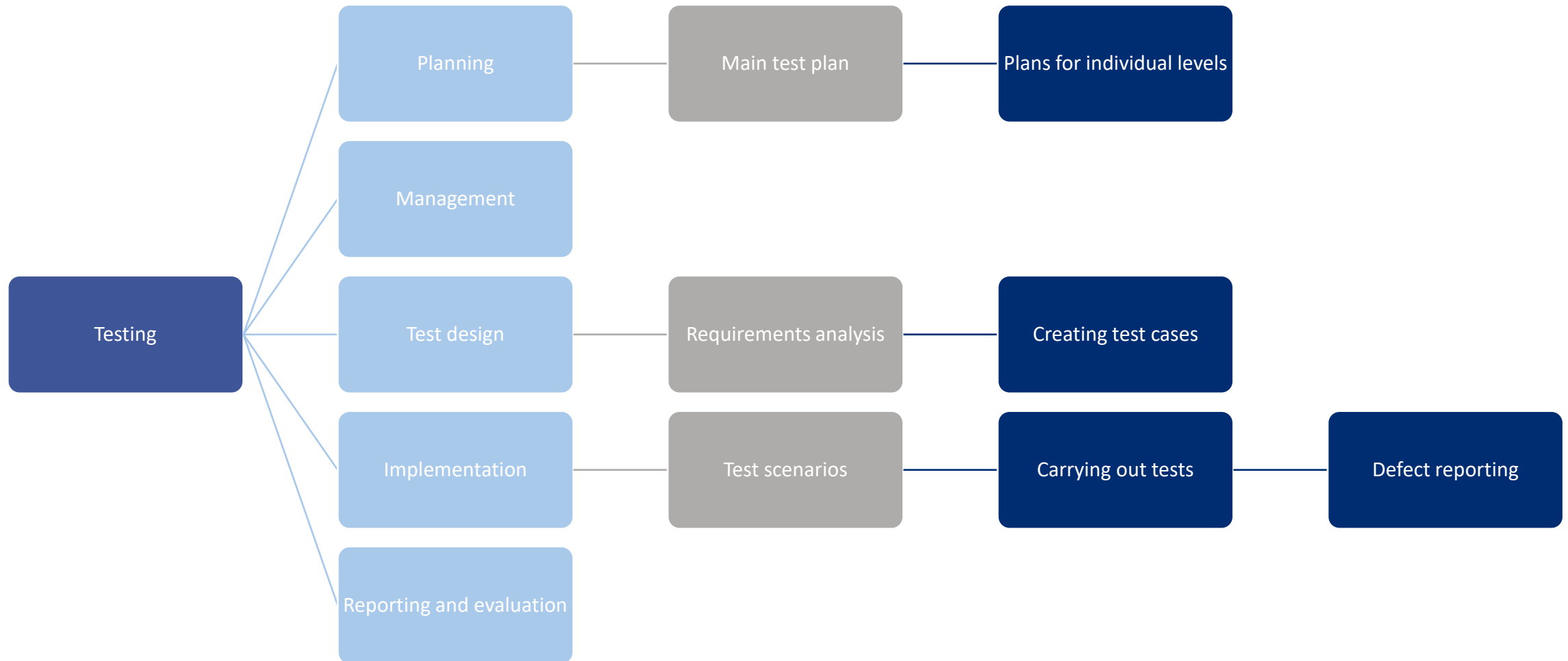


Division of tasks and activities

- detailed list of all tasks
- assigning a test team worker to a position
- clearly defined, independent, without overlap in action
- we define responsibility and the necessary human resources
- creating a detailed list of tasks and available resources
- creating a schedule of activities



Distribution example



Risks of the testing system

- It is important to determine which areas require increased attention due to a higher probability of error.
- identification is based on experience or consultation
- Typical risks:
 - changes in parts of the system – more modifications in the source code
 - changes or expansion of the system – in case of insufficient specifications
 - specific very complex functionality
 - system security and performance
 - interface to other systems – we don't even have to have control over the API development



Planning in an agile environment

- waterfall model
 - test plan created to satisfy external requirements – unnecessary paperwork
 - the plan is not up to date, not followed => the test plan is not very flexible
- agile approaches
 - Is it possible to plan testing effectively and successfully without a test plan?
 - progressive, dynamic planning
 - tests are not planned in advance
 - planning at the beginning of each iteration
 - highly flexible plan
 - tools, techniques and strategies are chosen according to current needs immediately before their development
 - are more accurate than forward planning based on specifications
 - a new test plan is created for each iteration
 - if required
 - we focus on unchanging processes – method and rules for reporting defects, required test coverage, schedule, input and output criteria
 - after each iteration, the plan is updated and expanded with new information

Testing status report

- status reporting (*test reports*)
- content of the report:
 - clearly defined framework of tests performed and their results
 - monitored indicators and measurement results
 - summary of the overall situation and a clear conclusion
- a traffic light is also used to indicate
 - green – difference up to 10% compared to plan
 - yellow - the difference is no more than 25%
 - red – the difference is more than 25%

Example of a report

Názov projektu
verzia

Hlásenie stavu **SIT** testovania

Stav k dátumu	
Projekt a verzia	
Dátum vydania	
Trvanie fázy SIT	
Počet dní realizácie SIT	
Stav testovania	Zelený

Legenda stavu

Zelený	podmienky
Žltý	podmienky
Červený	podmienky
Iná	podmienky

Legenda dostupnosti

Zelený	podmienky
Žltý	podmienky
Červený	podmienky
Iná	podmienky

Testovaná oblasť (sada testov)	Začiatok		Ukončenie		Plán manuálnych testov				Defekty			Stav	
	Plán	Skutočnosť	Plán	Skutočnosť	Prešli	Neprešli	Nerealizovali sa	Nejde realizovať	Nové	Vyriešené	Uzatvorené	Dostupnosť implementácie	Testovanie
Oblasť 1												Zelený	Zelený
Oblasť 2												Zelený	Zelený
Oblasť 3												Zelený	Zelený
Oblasť 4												Zelený	Zelený
Oblasť 5												Zelený	Zelený
Oblasť 6												Zelený	Zelený
Spolu													



Defect reporting

- discovering the conflict between specification and actual behavior
- However, it doesn't always have to be a defect.
 - we are unable to initially correctly assess the nature and possible impact
- it is necessary to examine the behavior of the detected defect:
 - Are the internal and external data and settings correct?
 - Are we really working with the right version?
 - Did we proceed correctly, did we forget any action?
 - Can we repeat/demonstrate the manifestation of the defect?
 - If so, what is the easiest way?
 - Does the defect only appear under certain conditions, or is it general?
 - If it manifests itself in other situations, what are the impacts on the user?
 - Is it possible to prove the presence of a defect?

Example for reporting a defect

- A solution for displaying a list of documents on a page is being tested. The application contains approximately 35 documents created in various ways. In the standard view, 5 documents are displayed per page.
- Tester procedure:
 - It verifies that the list of the first 5 documents is displayed correctly and that clicking on any of them displays the clicked document.
 - Then, in the settings, change the number of documents per page from 5 to 20. Exit the settings and display the list of documents.
 - Instead of the list being displayed immediately, nothing is displayed/loaded - the application is unresponsive. This state lasts for more than a minute.
 - The tester must forcefully terminate the application.
- A defect was encountered – Does the tester have enough information to report?

Example for reporting a defect

- **No** , therefore the tester needs to spend the next few minutes reviewing the situation.
 - A document was discovered that is in the wrong format, and when loading it into the list, the list loading failed.
 - reporting can be more accurate and the problem of globally displaying 20 documents has been eliminated
 - Further investigation is underway to determine the defect:
 - Isn't the defect already known?
 - recording all events that are defective after this report, and reporting those as well
 - based on them it will be possible to prove that the error has been debugged

Defect reporting

- The tester's task is to discover any software defect and its relevant information, which needs to be processed and provided in the form of a report.
- Error/defect report (*defect report / incident report*)
 - created only if the given record does not exist
 - it is also created if it has not yet been possible to determine all the information, whether it is an error or a failure of the testing approach

When do we report a defect?

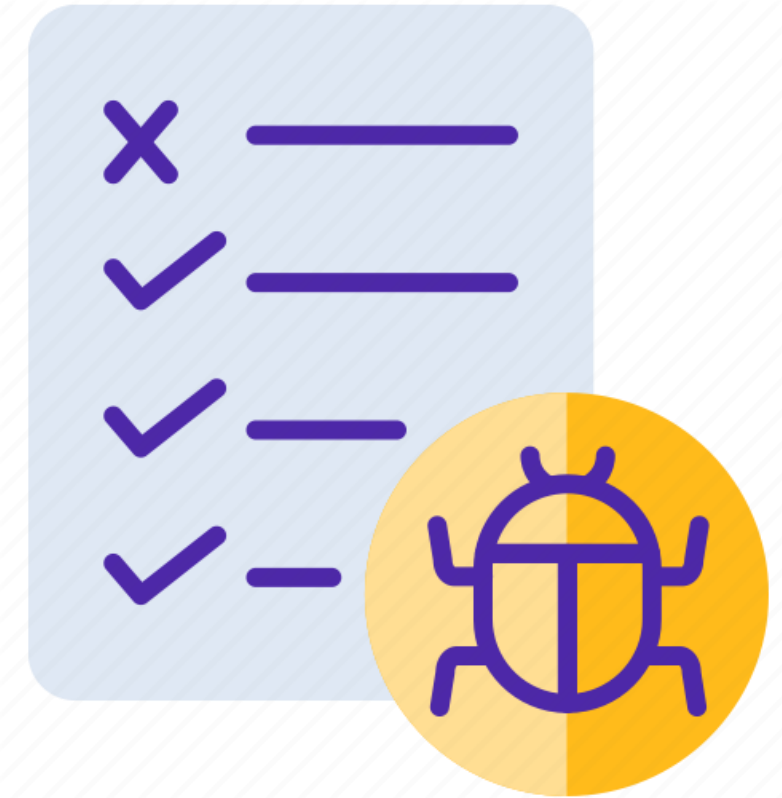
- the software does something it shouldn't or doesn't behave as it should
- the software does something that was not specified or required and may, according to the tester, be annoying to the customer
- the system user interface stops responding or is slow
- a specific system function becomes annoying very quickly – pop -ups
- a specific software component crashes
- System behavior is not consistent
- execution of specific functions is slower than in previous versions of the system
- the software is difficult to use and complex – by design
- the supporting materials for the project are out of date, incorrect or incomprehensible
- maintaining software as a product is more complicated than it should be

Defect reporting

- When reporting a defect, the tester must identify whether it is a new defect, or similar or related to already reported ones.
- in the case of an existing one, the report is checked and it can be expanded with new knowledge
- the decision whether it is a defect or a system function is typically made by the test analyst
- **defect is anything that reduces the economic value of a product.**
 - if the impact of the problem is more costly than the cost of fixing it
- A defect is considered to be any deficiency that violates the agreement between the customer and the supplier.
 - What qualities and behavior should he have?
- other deficiencies are processed as **change requests**
 - change request – new user story

Who are the reports for?

- defect reporting is an important output of the tester
- are intended in particular:
 - testing manager
 - development manager
 - analysts
 - other testers in the team
 - developers
 - customer representative
 - member of other teams
- the report can serve as a demonstration of an interesting or serious defect
- possibly as a deterrent example
- bug reports are not intended for private developer-tester communication only



Defect reporting form

- The report should be discussed at least with your supervisor.
- The report may include the time of occurrence, data used, screenshots
- A well-written report:
 - concise and precise
 - deals with only one defect (as long as it does not cause other functionalities to fail)
 - provides information about the data used and system configurations at the time of the test
 - describes or refers to the exact procedure for reproducing it
 - describes the expression as generally as possible
 - in the case of a graphic error, provide a description or image of the error
 - describes the impact of the defect, unless it can be deduced
 - provides additional relevant information as requested by the stakeholder
 - the content of the defect report and comments are formulated professionally and without emotion
 - visually, the report needs to be well formatted and clear so that the reader can quickly find what they need in the text

Example of a report

Summary: RTH 1.7.2. – User creation – User can be created without the mandatory field Add To Projects

Priority: 3 - Medium

Application: RTH 1.7.2.

Component: Administration

Environment: not relevant

Reporter: Anna Borovcová

Date reported: 1.5.2012

Attachment: C:\Projekty\ABC\chyby\bug54.png

Test case: Test case 23

Description: A user can be created without filling in the mandatory field "Add To Projects". The system submits the incomplete form and creates the user. It is not possible to log in to a user account created in this way.

Steps to reproduce: When creating a new user account, if the mandatory field "Add To Projects" is not filled in, the system displays an error message "System Warning: Invalid argument supplied for each" for a very short moment (for the full message, see the attachment). The user does not have a chance to read the error message because in less than 1 second, the application creates the new user account and displays the message "Operation Successfull". It is not possible to log in to a user account created in this way; the system reports the following to the user: "Warning: You have no default project. Please contact the system administrator."

Defect reporting management systems

- processing in a word processor or spreadsheet
- specialized tools are used in practice
- refer to such a system as *a defect/issue/bug tracking system*
 - Azure DevOps (Team Foundation Server), Backlog , Bugzilla , Mantis , Jira , Redmine
- is often part of test management systems
 - HP Quality Center, Practitest , Testpad , TestRail , TestBench , qTest
- The system should provide:
 - display of a short list of messages, filtering, sorting
 - attaching attachments
 - possibility of maintaining a larger number of comments
 - sending notifications when status changes
 - possibility of generating reports according to metrics
 - access via web browser

Dashboard of tool Xray

Atlas Mobile App

Sprint 1

QUICK FILTERS: [Only My Issues](#) [Recently Updated](#)

To Do

● ATLAS-3

↓ As a user, I want to reset my password so I can regain access when I forget it

1.0 - OK

2

● ATLAS-6

↓ As a user, I want to upload a profile picture so other users can recognise me easier

1.0 - UNCOVERED

1

● ATLAS-7

↓ As a user, I want set my profile's privacy so I can control what information is visible to other users

1.0 - UNCOVERED

3

● ATLAS-13

↓ As a user, I want to terminate my account so the App does not have any data about myself

1.0 - NOTRUN

3

In Progress

● ATLAS-4

↓ As a user, I want to login to my account with a username and a password so my account will be secured

1.0 - NOK

3

● ATLAS-5

↓ As a user, I want to edit my contact details so I can keep it up to date

1.0 - UNCOVERED

2

● ATLAS-11

↓ Test Plan for v1.0 Sprint 1

Done

● ATLAS-1

↓ As a user, I want to register myself so I can have an account

1.0 - UNCOVERED

5

● ATLAS-2

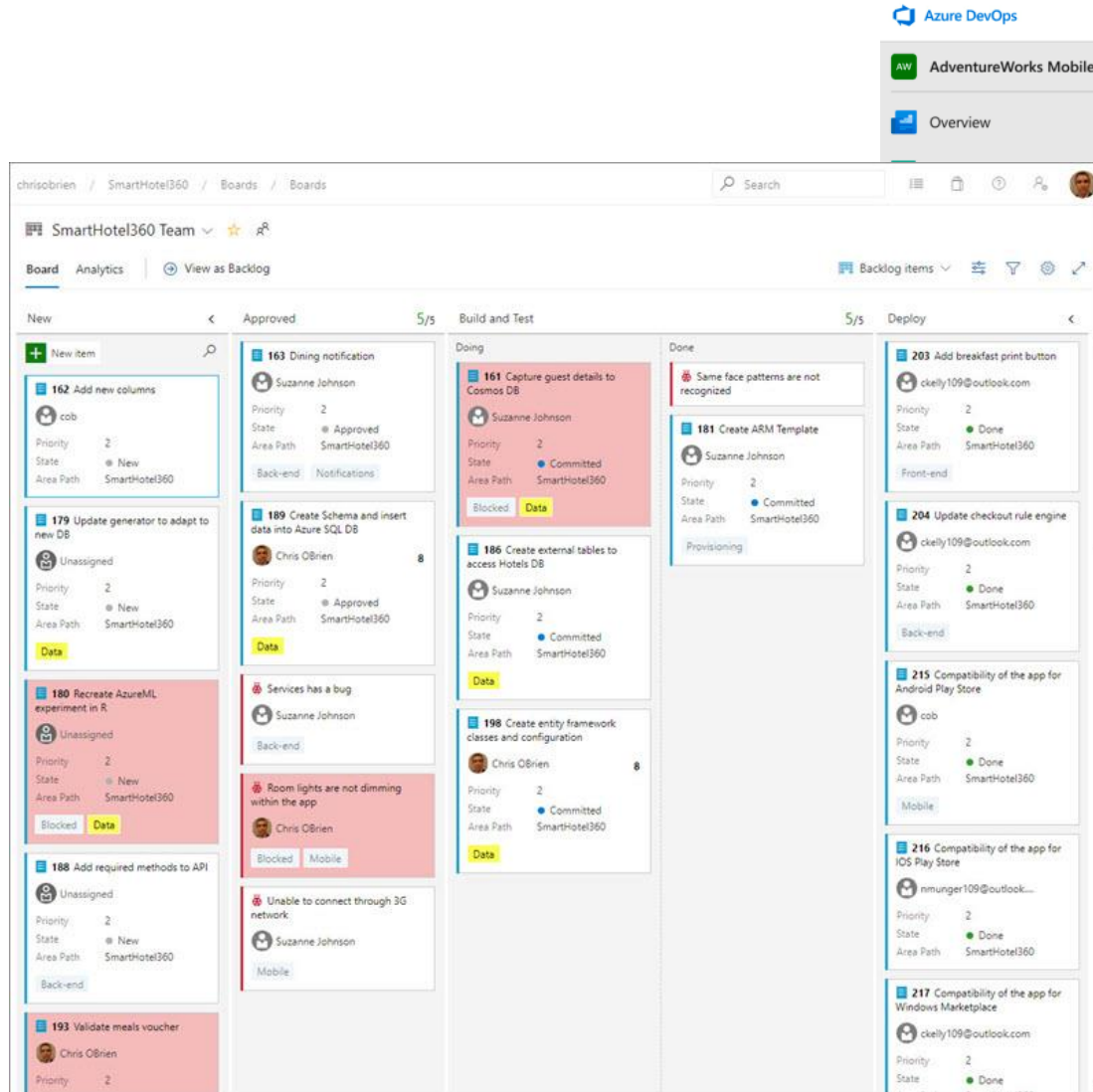
↓ As a user, I want to be enforced to have a strong password so my account is properly secured

1.0 - NOTRUN

2



Azure tool preview DevOps

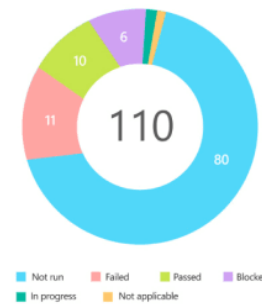


Contoso / AdventureWorks Mobile / Test Plans / Web Team

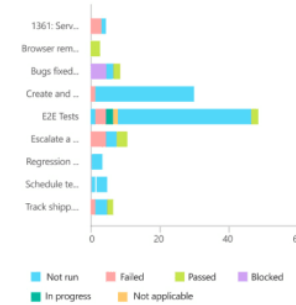
Test Plan: Web Team

Test Charts

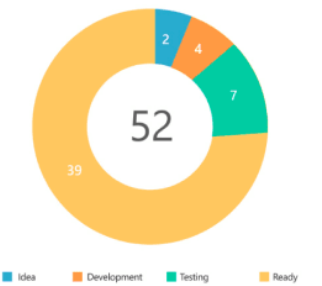
Overall Execution State



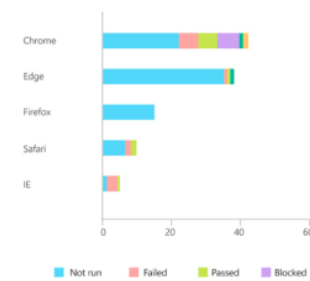
Tests by Suite



Test Automation Status



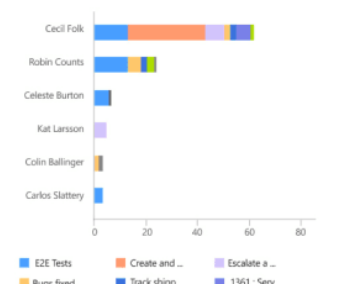
Configurations Coverage



Execution



Testing Ownership



Standard reporting elements

- it's not just one block of text
- multiple identifiers for easy filtering
- mandatory specified by IEEE 829-1998 standard – documentation of the test process
- Reporting elements:
 - **Unique identifier**
 - **Summary**: a brief description of the error in approximately 160 characters
 - **Priority and severity**
 - **Category**
 - **Information about the environment and settings** : including the version of the tested system
 - **Author** : the person who discovered the defect – another person can write
 - **Solver** : assigning a solution to a problem
 - **Status** : where the current message is located
 - **Description** : detailed description of the defect manifestation and related information
 - **Comments** : also a history of comments from different team members
 - **Attachments** : links to supporting documentation, logs, screenshots , emails from users
 - **Specific elements** : required in the report
 - **Creation date, resolution, and change history**



Defining relationships between reports

- each message describes only one defect
- is it possible to link two reports
- types:
 - **It is duplicated with XY**
 - The defect report exists twice. One report is closed as a duplicate.
 - **Is XY blocked/blocks XY**
 - indicates that a second defect needs to be resolved for testing
 - **Related to XY**
 - there is a certain connection between the two defects – common faulty implementation, same components...

Summary

- very brief explanation of the problem
- the reader must already know from the list what the report is about
- This includes information about:
 - name of the application and the given module
 - where exactly the symptoms of the defect occurred
 - During which action did the defect appear, what did the failure look like?
- Bad example:
 - Saving error
 - Go to the evil screen
 - The application crashed.
 - The screen does not accept numbers
- Examples of report summaries:
 - RTH 1.7.2 Deleted user can log in and make changes to XYZ records
 - STRI: When registering an account based on a power of attorney, it is possible to enter a proxy, even though it should not be possible
 - ATP prevents printing any document and reports error 205

Description

- provides all information about the test performed and the manifestations of the defect that are relevant to assessment and repair
- brief in case of a simple defect
- extensive description in case of complications
- contains:
 - recapitulation of information from the summary with more detailed information
 - minimum necessary sequence of steps to reproduce the manifestation of the defect
 - an explanation of what exactly is considered a defect
 - description of the manifestation and error messages, screenshots
 - informing when a defect occurs and when it does not
 - description of possible relationships to other defects
 - further information, depending on the type of project

Description - example

- avoid vague descriptions:
 - "Campaign emails are also being sent **to some** clients who don't have them. At the same time, for **some** clients, **sometimes** not all the information is displayed in the details. I think there is a connection."
- example description:
 - When creating a new campaign and subsequently sending an email to clients' email addresses, the email is also sent to unsubscribed clients.
 - This error only occurs with clients that were logged out before the new system version (7.2) was deployed. The error does not occur with clients that logged out after the new version was deployed, e.g. as part of a test.
 - **Data :**
 - Example of previously logged out clients: client XYZ@xxx.sk, Client12435@xxx.sk
 - Example of newly logged out clients: nas.test@xxx.sk
 - **Defect simulation procedure:**
 - From the Menu, click New Campaign.
 - Fill out the form with valid values.
 - We will select the clients to whom the campaign should be sent, so that they include those who have previously
 - logged out, newly logged out and active clients.
 - Active clients (correct) and those who have previously unsubscribed (wrong) will receive the email. New clients will not receive the email.
 - logged out clients (correct).
 -
 - **Connections with another defect:**
 - This error is likely related to error 457: "Information is not displayed about clients that have not been changed by the 7.2 deployment"

Attachments

- one of the elements that should be used according to the needs of the report readers
- we use in case of defects that are:
 - difficult to reproduce
 - of a graphic nature
 - their presence is indicated by entries in the system log
 - exceptional reports
- The attachment may not only be a demonstration of the defect, but also as a sample of the test data used.

Attachments - example

Elements above each other are offset differently

Parametry komunikace

Spojení

Typ spojení Přímé (auto) Použít HW klíč

Šifrování 128 bit Aktivní od: 16.5.2012 15

Cílové body

Lokace Londýn, Liverpool

Uzavřít kanál po 30 s
Uzavřít po 60 s

Poznámka
Lokace k tomuto bodu odp

Storno Uložit

The items in the combobox are not sorted alphabetically (according to the manual they should be)

The text is not wrapped and the scroll bar is missing.

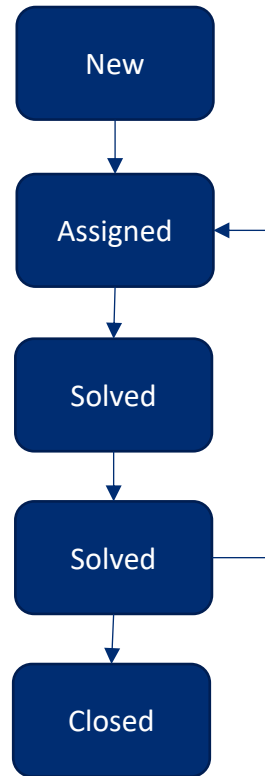
The button is inactive even if all required fields are filled in.

Irreproducible defects

- There are cases where the discovered defect is clear, but by repetition, we cannot repeat the occurrence.
- arise from a certain complex combination of actions taken
- sometimes the same steps are performed and the defect does not manifest itself
- these pseudo-random behaviors of defects are caused by certain conditions that are not fully known
 - internal state of the system, components
 - environmental condition
 - external factors
- an unreproducible defect becomes reproducible when we know all the conditions
- the report will indicate that it is an unreproducible defect
- the frequency of occurrence and times are recorded
- it is thoroughly documented – screenshots , video, log extract

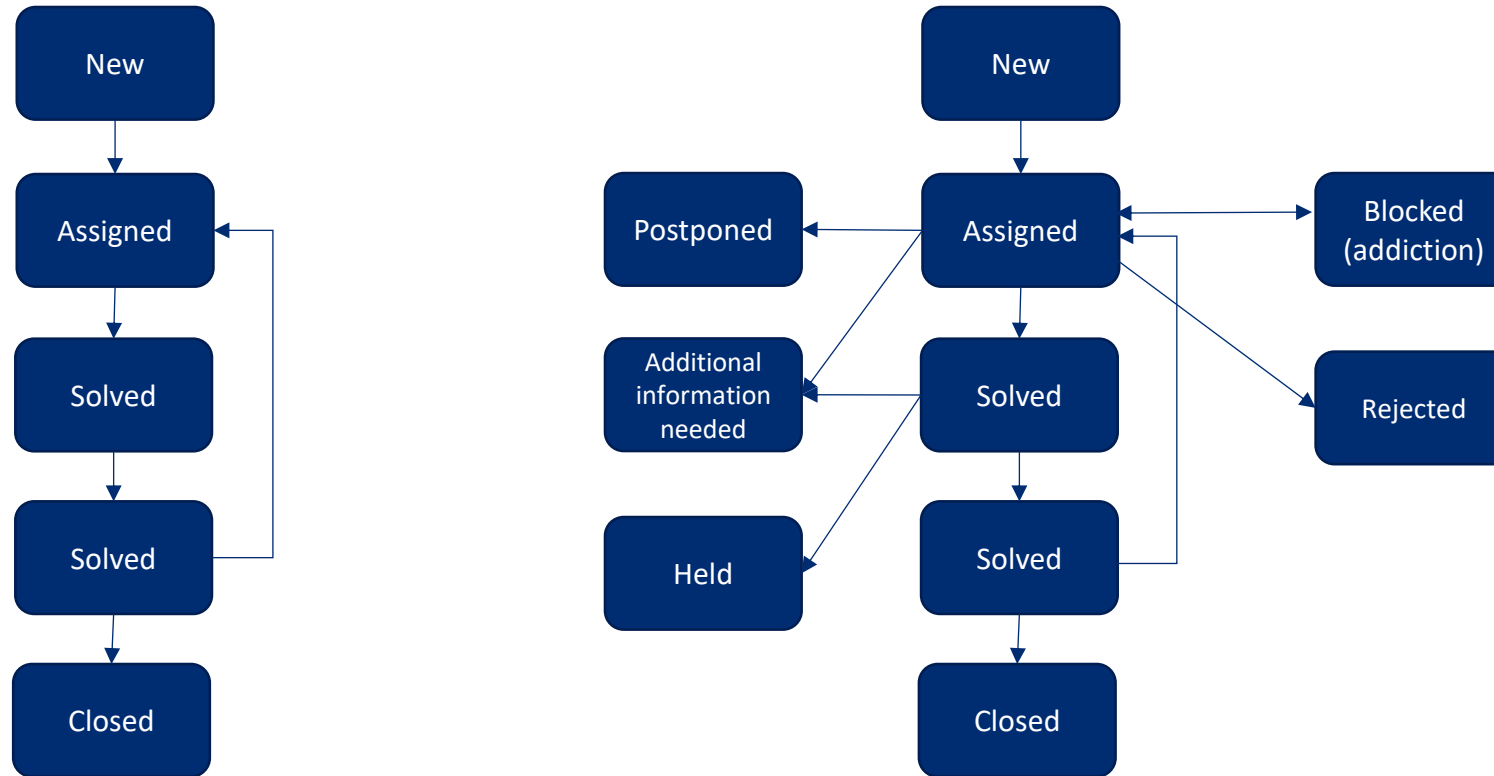
Lifecycle of a reported defect

During the existence of a report, it will go through several states from identification to its closure



Lifecycle of a reported defect

During the existence of a report, it will go through several states from identification to its closure



How people reacts differently to a
single word.

"Bug"



Tester



Developer



Manager

Thank you for your attention

doc. Ing. **Jozef Kostolný**, PhD.
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
jozef.kostolny@fri.uniza.sk